

# 1. Introduction

Par groupe de 3, nous devons développer un jeu intégré à un site web, nous avons décidé de développer un Pac-Man.

L'objectif était donc de créer un jeu de course dans un labyrinthe avec ennemi(s) essayant de piéger le joueur en trouvant le chemin le plus court. Afin que le jeu donne envie de continuer ou de s'améliorer nous avons créé un score qui donne des points en fonction des points manger par le joueur.

---

## 1. Description des fonctionnalités

Notre jeu permet à l'utilisateur de jouer en augmentant son score grâce à des points manger par le joueur, les points font gagner 10 points au score.

Sur le jeu, on peut voir deux cartes au choix.

### 2.1 Fonction tri

Ainsi, l'utilisateur peut rechercher des films selon les critères suivants : le titre, la date de production, le genre et la note du public. Il peut sélectionner un ou plusieurs critères à la fois.

De la même façon, s'il souhaite que les résultats soient triés en fonction de leur popularité et leur revenu, il peut sélectionner "Popularity et Revenue" dans la liste de choix "Tri" du menu principal.

### 2.2 Fonction recherche

L'application va alors lui proposer des films correspondant à sa recherche et trier selon le choix de l'utilisateur.

L'utilisateur peut utiliser des films simples comme le genre, l'année, le pays, la note ou la langue du film. Il peut également utiliser des critères de recherche partiels sur le titre ou le titre original du film et le nom du réalisateur.

### 2.3 Fonctions globales

Le bouton « Réinitialiser » demande à la fonction « Réinitialiser » de recharger les données initiales et les afficher dans le tableau résultat.

Le bouton « Exporter » demande à la fonction « exported\_movies » d'écrire le tableau de résultats dans un nouveau fichier csv accessible dans le répertoire data et lisible par l'utilisateur.

### 2.4 Top 10

L'utilisateur souhaite que le programme lui propose uniquement les 10 films au revenu le plus élevé pour un genre donné. Pour obtenir ce résultat, il peut cliquer sur le bouton "valider" du menu nommé "Top 10 des films par revenu pour un genre" dans le second menu.

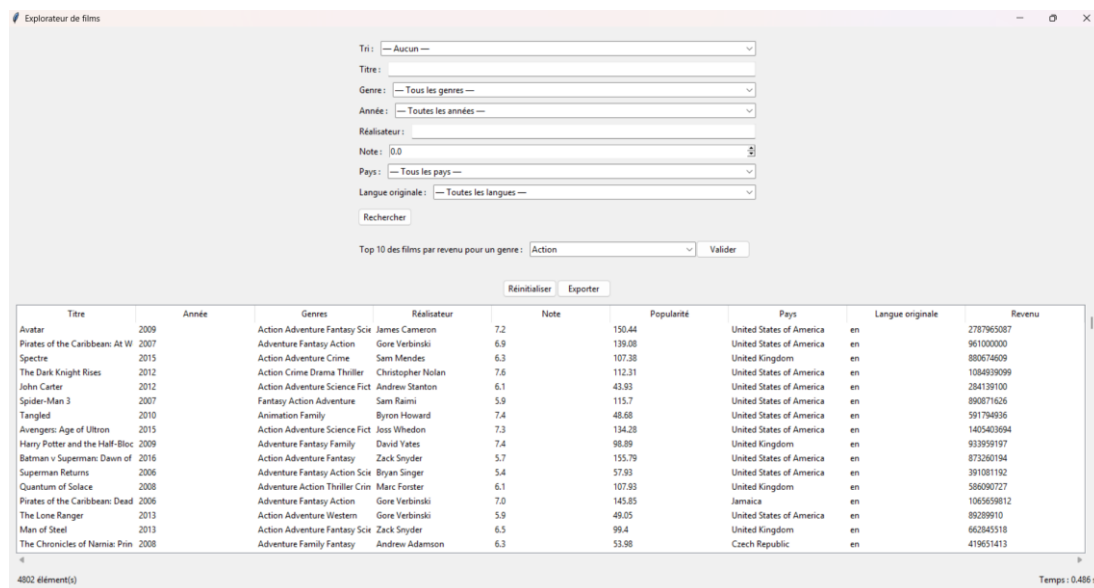
### 2.5 Tableau résultat

Les films sont affichés dans un tableau de plusieurs colonnes. La première colonne est pour le titre et titre original des films, il y en a ensuite une pour les années de sortie, puis une autre pour le genre du film. Par la suite il y a une colonne pour le réalisateur, puis une autre pour afficher la note de celui-ci, il y en a ensuite une qui montre la popularité. Il y en a également une pour le pays du film, ensuite il y a une colonne pour donner la langue originale et enfin une dernière qui affiche les revenus tirés du film.

## 2.6 Barre d'état

Après chaque exécution d'une fonction, la barre d'état donne le nombre d'élément qu'il y a dans le tableau et montre également le temps de recherche.

## 2.7 Capture d'écran



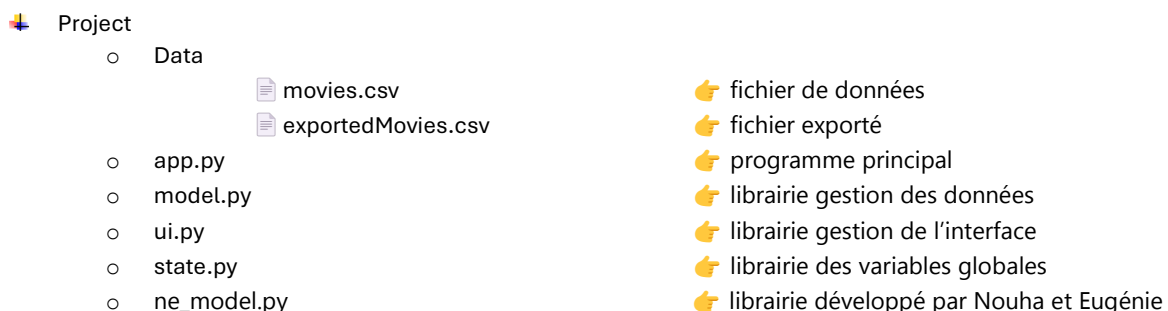
## 3 Choix de conception

Nous avons choisi l'architecture recommandée dans les consignes (spécifications).

Les fonctions de recherche, de tri, d'ouverture et de transformation du tableau de films sont dans le fichier `model.py`, les appels des fonctions et la mise en page de l'interface Tkinter sont dans `ui.py` et le programme principal de l'application est dans `app.py`.

Les données sont stockées dans le fichier `movies.csv` du répertoire `data`.

### Schéma du dossier projet.



### 3.1 Structure des données

Ce fichier au format CSV (Comma Separated Values) contient les données des films sous forme de chaînes de caractères avec entre autres :

- Titre
- Titre original
- Date de production

- Genre
- ...

### 3.2 Structure de la table des films et de résultats en mémoire

Tableau de dictionnaires dont chaque entrée contient les données du film sous forme clé-valeur.

### 3.3 Gestion des champs particuliers

Au démarrage, le tableau est nettoyé après la lecture du fichier afin de retirer les entrées non conformes. Il fallait supprimer les films si leur date de production n'avait pas été renseigné dans le fichier.

De plus, ces dates de production, auparavant sous forme de chaînes de caractères "YYYY-MM-DD", ont été transformées en conservant uniquement l'année "YYYY". De la même manière, le revenu et la note de chaque film a été convertis en décimal afin de pouvoir comparer chaque revenu et de les trier dans le programme. Cependant les genres de chaque film étaient sous forme d'une unique chaîne de caractères. Par conséquent, il fallait extraire tous les genres distincts et en dresser une liste. Cela était nécessaire pour créer la combobox utile à l'interface utilisateur. Il peut alors sélectionner les genres pour obtenir des films correspondants.

---

## 4 REPARTITION DES TACHES (TRIO)

Nous nous sommes réparties les tâches de sorte que notre travail individuel soit différent pour chacune. Ainsi, chaque personne était responsable des fonctions de son module, sauf pour Nouha qui a intégré sa fonction `load_movies()` au module `model.py`.

Tout d'abord Nouha devait écrire la fonction `load_movies()` qui permettait d'ouvrir le fichier et de mettre son contenu dans un tableau. Les conversions et la normalisation des genres se faisaient également dans cette fonction par l'appel de quatre fonctions tierces depuis `load_movies()`; avec `normalize_genres()`, `normalize_release_dates()`, `normalize_popularity()` et `normalize_revenue()`.

Eugénie a réparti les tâches en décrivant les différentes fonctions par leur nom, leurs paramètres et le format des valeurs renvoyé dans `model.py`. Elle devait également écrire les fonctions de recherche et de tri. La fonction de recherche prend en paramètre un dictionnaire contenant les critères de recherche demandés en clés et les recherches entrées par l'utilisateur en valeurs. Cette fonction renvoie le tableau de résultats contenant les films correspondants aux recherches qui apparaîtra ensuite dans l'interface. Les fonctions de tri prennent en paramètre le tableau de résultats et les trient en fonction de leur revenu, leur popularité ou renvoient uniquement les 10 premiers films au revenu le plus élevé.

Ambre devait créer l'interface Tkinter qui permet à l'utilisateur d'entrer ses recherches et d'afficher les résultats. Elle a dû reprendre les développements réalisés par ses camarades et les adapter pour que l'interface utilise les fonctions du modèle. Elle a également ajouté un bouton exporter qui permet d'utiliser la fonction « `exported_movies` » pour générer le dossier. Enfin elle a utilisé le rapport partiel de ses camarades qu'elle a complété pour obtenir ce rapport final.

## 5. DIFFICULTES RENCONTREES ET SOLUTIONS APORTEES

Pour la partie de Nouha, les difficultés rencontrées sont de savoir comment faire et par quoi commencer. Elle a fait la lecture du CSV et ses transformations. Elle n'a pas eu de problème avec l'ouverture du fichier mais elle en a eu avec les transformations. Elle devait créer un tableau qui prenait uniquement les lignes avec un titre. Elle ne savait pas comment faire pour toutes les autres conditions et les transformations des données. Elle a donc demandé à son père de lui expliquer comment faire, pour chaque film :

Transformation de la date, elle a donc converti « release\_date » pour ne prendre que les 4 premiers caractères qui correspondent à l'année

Popularité, elle a converti la valeur en nombre flottant avec la fonction « float »

Ensuite pour le revenu, elle a converti la valeur du fichier en entier

Pour finir, elle a filtré sur le titre pour exclure les films sans titre.

Pour Eugénie, au début, la fonction de recherche faisait plus de 8 lignes par critères car elle enchainait des conditions « if..else ». Elle a fini par se rendre compte qu'une variable booléenne, nommée "ajouter", permettait de tester chaque critère et de modifier cette variable successivement et ainsi réduire le code à une condition par critère et éviter des imbrications de « else ». Ainsi pour chaque critère, s'il est présent en clé du dictionnaire, le résultat pour ce critère est accumulé grâce à l'opérateur booléen AND (ET) à la variable « ajouter ». Lorsque la condition est respectée (que la recherche de l'utilisateur correspond au contenu de la ligne parcourue par le programme), la variable "ajouter" reste vrai. Si une condition n'est pas respectée, la variable "ajouter" bascule à faux. Si la variable reste vraie jusqu'à la fin des tests sur les critères alors le film est ajouté au tableau des résultats.

Pour la partie de Ambre, la principale difficulté a été d'utiliser le code de ses camarades. En effet, les fonctions développées pour l'interface ne s'adaptaient pas aux fonctions développées dans le modèle. Elle a dû modifier certaines fonctions pour correspondre aux contraintes de l'interface. La 2nd difficulté fut la gestion des composants graphiques qu'elle ne connaissait pas. Elle a demandé à son père comment faire. Il l'a aidé à comprendre le fonctionnement et à utiliser les fonctions de la librairie « Tkinter », notamment la notion de cadre graphique.

Elle a utilisé le logiciel de développement vscode car il intègre une aide au développement avec l'IA, ce qui a simplifié la programmation de l'interface.

---

## 6. Pistes d'amélioration

Plusieurs améliorations pourraient être envisagées :

Il aurait été préférable de passer les variables comme "films" en paramètres des différentes fonctions plutôt que de les partager en variables globales. Ambre a eu des difficultés lors de la séparation en plusieurs, elle a dû créer une librairie particulière pour gérer les variables globales entre plusieurs fichiers.

- Les variables globales peuvent également engendrer des effets de bords dans l'application
- Ajouter un système de tri direct en cliquant sur les en-têtes du tableau
- Pour faire patienter l'utilisateur pendant l'exécution des fonctions, on aurait pu intégrer une barre de progression
- Améliorer le design de l'interface

---

## Conclusion

Ce projet nous a permis de mettre en pratique de nombreuses notions vues en NSI : structures de données avec les listes, tableaux et dictionnaires. Pour filtrer et rechercher, nous avons utilisé les conditions, les boucles. Enfin, nous avons découvert la librairie « Tkinter ».

Le travail en groupe nous a également appris comment mieux répartir les tâches et collaborer.